SOLUZIONE DEL TEMA: 3 febbraio 2020



Politecnico di Milano

Anno accademico 2018-2019

Ingegneria del Software

Cognome:		LAUREANDO ☐
Nome:	Matricola:	
Sezione (segnarne una): ☐ Cugola	☐ Margara	☐ San Pietro

Istruzioni

1. La mancata indicazione dei dati anagrafici e della sezione comporta l'annullamento del compito.
2. Al termine, consegnare solo i fogli distribuiti utilizzando il retro delle pagine in caso di necessità. Non separare questi fogli. Eventuali fogli di brutta, ecc. non verranno in nessun caso presi in considerazione. È possibile scrivere in matita.
3. È possibile consultare liberamente libri, manuali o appunti. È proibito l'uso di ogni dispositivo elettronico (quali calcolatrici tascabili, telefoni cellulari, smartwatch, ...).
4. Non è possibile lasciare l'aula conservando il tema della prova in corso.
5. La consegna di un compito gravemente insufficiente comporta l'impossibilità a partecipare all'appello successivo.
6. Tempo a disposizione: 2h.

Esercizio 1:

Esercizio 2:

Esercizio 3:

Esercizio 4:

Esercizio 1

Si consideri la seguente specifica di un database di tratte servite da una compagnia area.

```
public class FlightsDB  {
  // \result==true sse esiste una tratta tra from e to
  public /*@ pure @*/ boolean directlyConnected(String from, String to);

  // \result==true sse esiste una tratta tra from e to oppure
  // una sequenza di tratte con scali intermedi tra from e to
  public /*@ pure @*/ boolean connected(String from, String to);

  // aggiunge la tratta al database
  // lancia InvalidArgumentException se to e from sono lo stesso aeroporto
  public void addRoute(String from, String to) throws InvalidArgumentException;

  // ritorna l'insieme delle destinazioni che sono raggiungibili da from con due
  // tratte consecutive (un solo scalo) ma non mediante una tratta diretta
  public /*@ pure @*/ Set<String> reachableWithOneStop(String from);
}
```

Domanda a)

Si specifichino in JML i metodi `connected`, `addRoute` e `reachableWithOneStop`.

Soluzione

Definiamo

```
//@unchanged() = (\forall String x; x!=null;
//@                (\forall String y; y!=null && !x.equals(y);
//@                  connected(x, y) <==> \old(connected(x, y)) &&
//@                  directlyConnected(x, y) <==> \old(directlyConnected(x, y)) ) )
```

```
//@requires from!=null && to!=null
//@ensures \result <==> (directlyConnected(from, to) ||
//@  (\exists String x; x!=null; connected(from, x) && directlyConnected(x, to)) )
```

```
public /*@ pure @*/ boolean connected(String from, String to);
```

```
//@requires from!=null && to!=null
//@ensures !from.equals(to) && directlyConnected(from, to) &&
//@  (\forall String x; x!=null;
//@    (\forall String y; y!=null;
//@      directlyConnected(x, y) <==>
//@        (\old(directlyConnected(x, y)) || (x.equals(from) && y.equals(to))) ))
//@signals(InvalidArgumentException) from.equals(to) && unchanged()
```

```
public void addRoute(String from, String to) throws InvalidArgumentException;
```

```
//@requires from!=null
//@ensures !\result.contains(null) &&
//@  (\forall String x; x!=null && !x.equals(from); \result.contains(x) <==>
//@    (!directlyConnected(from, x) &&
//@      (\exists String y; y!=null && !y.equals(x);
//@        directlyConnected(from, y) && directlyConnected (y, x) ) ) )
```

```
public /*@ pure @*/ Set<String> reachableWithOneStop(String from);
```

Domanda b)

Si consideri la seguente implementazione (parziale) di `FlightsDB` che si basa su una mappa che associa a ogni aeroporto l'insieme degli aeroporti connessi mediante una tratta diretta.

```
public class FlightsDB {
  private Map<String, Set<String>> connections = new HashMap<>();
  public boolean directlyConnected(String from, String to) {
    return connections.containsKey(from) && connections.get(from).contains(to);
  }
  public boolean connected(String from, String to) {
    return directlyConnected(from, to) || (connections.containsKey(from) &&
       connections.get(from).stream().anyMatch(x -> connected(x, to) );
  }
  public void addRoute(String from, String to) throws InvalidArgumentException {
    if (from.equals(to)) throw new InvalidArgumentException();
    if (!connections.containsKey(from)) connections.put(from, new HashSet<>());
    connections.get(from).add(to);
  }
  public Set<String> reachableWithOneStop(String from) { .. }
}
```

Si fornisca un invariante di rappresentazione per tale implementazione.

Soluzione

```
//@private invariant connections!=null && !connections.containsKey(null) &&
//@  (\forall String x; connections.containsKey(x);
//@      connections.get(x)!=null && !connections.get(x).contains(null) &&
//@      !connections.get(x).contains(x) )
```

Domanda c)

Si consideri una classe `NewFligthsDB` che permette di invocare il metodo `addRoute` con due parametri uguali senza lanciare eccezioni. La classe `NewFligtsDB` può essere definita come erede di `FligthsDB` rispettando il principio di sostituzione di Liskov? Si consideri invece una classe `RemovableFligthsDB` che aggiunge un metodo `remove` per rimuovere una tratta. La classe `RemovableFligtsDB` può essere definita come erede di `FligthsDB` rispettando il principio di sostituzione di Liskov? Motivare le risposte.

Soluzione

La classe `NewFligthsDB` non può essere definita come erede di `FligthsDB`, ad esempio perché non rispetterebbe la post-condizione eccezionale del metodo `addRoute`: un utilizzatore si attenderebbe un'eccezione che non viene sollevata.

Nemmeno la classe `RemovableFligthsDB` può essere definita come erede di `NewFligthsDB` perché viola la proprietà evolutiva che impedisce a una tratta di essere rimossa: un utilizzatore potrebbe stupirsi nel non ritrovare una tratta inserita.

Esercizio 2

Si consideri la seguente interfaccia Java. Il metodo `acquire` può essere invocato al più 10 volte prima di sospendere il chiamante. Il metodo `reset` re-inizializza l'oggetto, permettendo altre 10 invocazioni di `acquire` (incluse quelle già effettuate che erano in attesa al momento dell'invocazione di `reset` e vanno sbloccate).

```
public interface MultiLock {
  /* Sospende il chiamante se vi sono state più di 10 invocazioni precedenti
   * del medesimo metodo (dall'ultima invocazione di reset) */
  public void acquire() ;
  /* Re-inizializza l'oggetto permettendo altre 10 invocazioni di acquire
   * (incluse quelle già effettuate che erano in attesa) */
  public void reset() ;
}
```

Domanda a)

Si fornisca una implementazione per l'interfaccia `MultiLock`.

Soluzione

```
public class MultiLockImpl implements MultiLock {
  private int numAcquired;
  public MultiLockImpl() {
    numAcquired = 0;
  }
  public synchronized void acquire() {
    while(numAcquired>=10)
      try { wait(); }
      catch(Exception ex) {ex.printStackTrace();}
    numAcquired++;
  }
  public synchronized void reset() {
    numAcquired=0;
    notifyAll();
  }
}
```

Domanda b)

Si consideri il seguente frammento di codice che usa un oggetto di tipo `MultiLock`:

```
final MultiLock lock = new ...
int numAcquire = ...;
int numReset = ...;
for(int i=0; i<numAcquire; i++) {
  new Thread( () -> lock.acquire() ).start();
}
for(int i=0; i<numReset; i++) {
  new Thread( () -> lock.reset() ).start();
}
```

Quali valori sono ammissibili per le variabili `numAcquire` e `numReset` affinchè tutti i thread creati possano prima o poi procedere senza che nessuno rimanga sospeso?

Soluzione

Poichè non vi è garanzia sull'ordine di esecuzione dei thread creati dal frammento di codice, l'unica possibilità affinchè tutti possano procedere è che `numAcquire` sia minore o uguale a 10, indipendentemente dal valore di `numReset` (i thread che eseguono il metodo `reset` potrebbero partire tutti prima dei thread che eseguono `acquire`).

Esercizio 3

Si consideri il codice del seguente metodo, in cui ogni istruzione è numerata per comodità:

```
1  static int mystery(int x, int y) {
2    if (x>=0 && y>0) {
3      do {
4        if (x >= 4)
5          y++;
6        else
7          y--;
8        x--;
9      } while (x!=1);
10   }
11   if (y==0)
12     y=1;
13   return y;
14 }
```

Domanda a)

Qual è il numero minimo di dati di test necessari per coprire tutte le decisioni e le condizioni del programma? Si mostri un esempio, motivandone la minimalità.

Soluzione

Una volta entrati nel ciclo, ossia con $x >= 0$ e $y > 0$, basta $x >= 4$ per entrare nel ramo `then` dell'`if`. Il ciclo viene eseguito almeno altre tre volte, per uscire con $x == 1$, coprendo anche il ramo `else` dell'`if`. Ad esempio, basta il caso `(4, 1)`.

Occorre ancora coprire il caso false del primo `if`, rendendo falsa la prima condizione e rendendo falsa la seconda condizione. Serve dunque un caso di test con $x < 0$ e un caso di test con $y <= 0$: ad esempio `(-1,1), (0,0)`. Questi casi di test valutano anche in modo negativo e positivo la condizione dell'`if` alla riga 11.

Non è possible ridurre ulteriormente il numero di casi, in quanto per coprire le due condizioni all'interno del primo `if` sono comunque necessari tre casi. Dunque tre casi è il numero minimo.

Domanda b)

Si consideri il cammino seguente: 1, 2, 3, 4, 5, 8, 9, 4, 7, 8, 9, 4, 7, 8, 9, 10, 11, 12, 13. Cacolare le condizioni perché sia eseguito (path condition) e si scriva un caso di test per percorrerlo.

Soluzione

Il ciclo è eseguito 3 volte (quindi $x = 4$), incrementando y una volta e decrementandolo due volte, per terminare con $y = 0$ (per eseguire la riga 12). Quindi $y = 1$.

Condizione: $x == 4$ && $y == 1$, quindi caso di test `(4,1)`.

Esercizio 4

Si consideri il seguente metodo statico Java, scritto in stile imperativo:

```java
public static int deepMistery(final String name, final List<String> names) {
  if (names==null || name==null) return 0;
  int count=0;
  for (String x : names) {
    if (name.equals(x)) count=count+2;
    else count--;
  }
  return count;
}
```

Domanda a)

Scrivere lo stesso programma in stile funzionale, completando il codice sequente negllo spazio lasciato bianco:

```java
public static int deepMistery(final String name, final List<String> names) {
  if (names==null || name ==null) return 0;









  }
}
```

Soluzione

```java
public static int deepMistery(final String name, final List<String> names) {
  if (names==null || name ==null) return 0;
  return names.stream()
              .map(x -> ((x.equals(name)) ? 2 : -1))
              .reduce(0, (subtot, elem) -> subtot + elem);
  }
}
```

Oppure

```java
public static int deepMistery(final String name, final List<String> names) {
  if (names==null || name ==null) return 0;
  return names.stream()
              .mapToInt(x -> ((x.equals(name)) ? 2 : -1))
              .sum();
  }
}
```